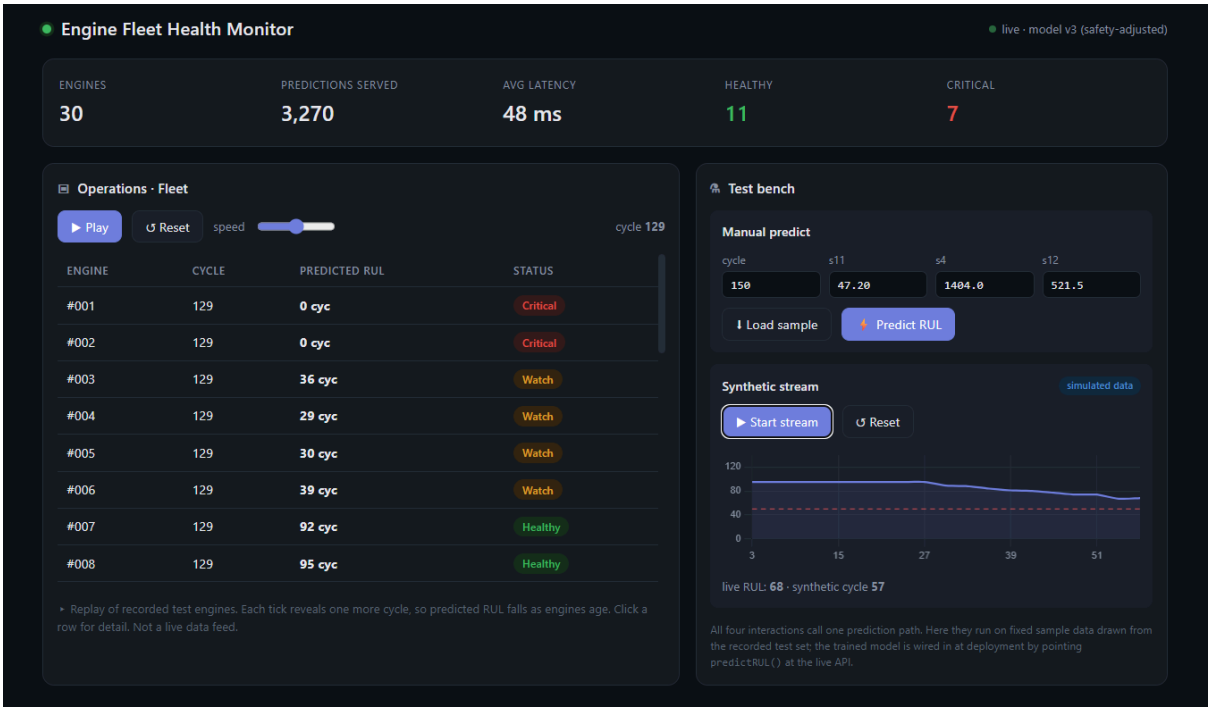
AEPM - Deployment Ideas (Monitoring Dashboard)

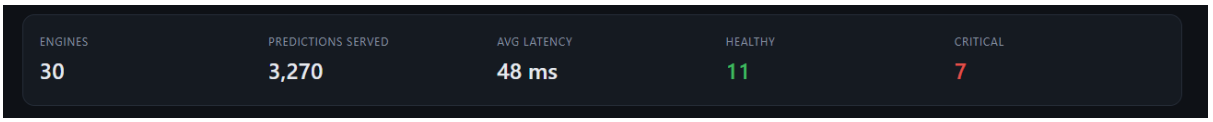
The overall app

web page that turns our trained model into something people can watch and use. It has three zones: a strip of numbers across the top, an "Operations" area on the left, and a "Test bench" on the right. Everything on the page ultimately asks the same model one question — "how many cycles does this engine have left?" — just in four different ways



.

1. The monitoring strip (top)



What it shows: five live numbers — how many engines are being watched, how many predictions the system has made, how fast it's responding (latency in milliseconds), how many engines are healthy, and how many are critical.

What it's for: it's the system's own vital-signs monitor. It proves the deployment is alive and tracking itself — which is exactly what the project's monitoring requirement asks for. It's always visible, no matter what else you're doing, like the heart-rate monitor in a hospital room.

2. Operations — Fleet view (left, the landing screen)

What it shows: a table of engines, each with its current cycle, predicted RUL (cycles remaining), and a colored status tag — green (healthy), amber (watch), red (critical). A Play button advances time; a speed slider controls how fast.

What it's for: this is the maintenance team's main screen — at a glance you see which engines need attention. When you press Play, time moves forward and you *watch* engines age: their RUL ticks down and they shift from green to amber to red. This is the standout, demo-friendly feature, and it ties straight to the business goal (catch failures before they happen).

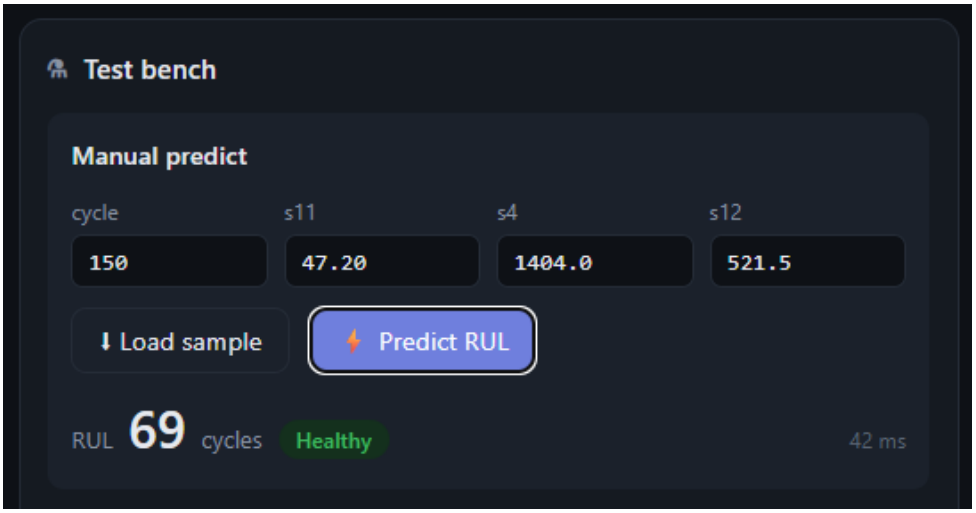
Important: this is a *replay* of recorded test engines — real data revealed gradually — not a live feed.

3. Operations — Engine detail (opens when you click a row)

What it shows: zoom into one engine. You get its current predicted RUL and cycle, a line chart of RUL falling over time toward a red "maintenance threshold" line, a second chart showing two key sensors drifting (s11 rising, s4 falling as it degrades), and a plain-language recommendation ("schedule maintenance within X cycles").

What it's for: this is the "why." Instead of just a number, it shows the *reasoning* — the sensor trends driving the prediction. That builds trust and is your interpretability story (the kind of transparency the field values). Click "Fleet" to go back.

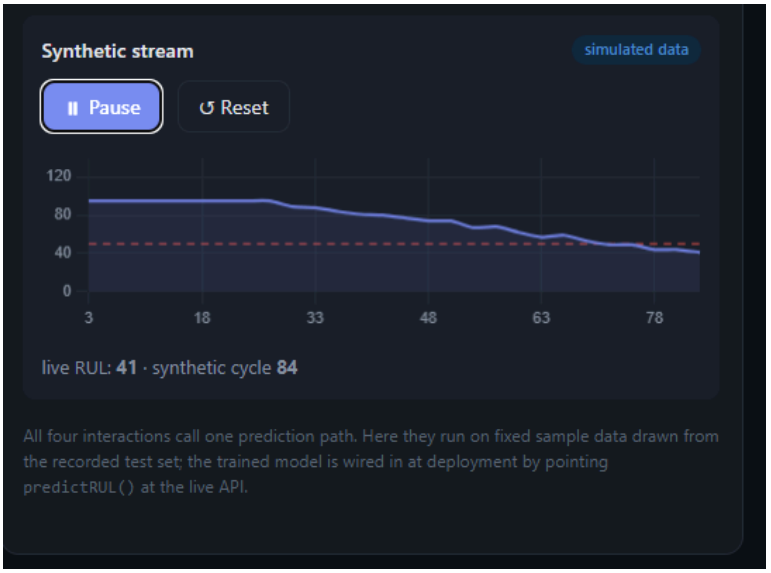
4. Test bench — Manual predict (right, top box)



What it shows: a small form with input fields (cycle, a few sensors), a "Load sample" button to autofill realistic values, and a "Predict RUL" button. Hit predict and you get an RUL number, a status tag, and the response time.

What it's for: the hands-on "try it yourself" tool. It proves the model works on *any* input someone gives it — not just the canned replay. At your demo, a grader can hand you values and watch the model answer live. This is genuine live inference once the backend is connected.

5. Test bench — Synthetic stream (right, bottom box)



What it shows: press Start and a chart draws live — a made-up engine that degrades cycle by cycle, its predicted RUL plotted falling toward the threshold, with a "Simulated" badge in the corner.

What it's for: it demonstrates the system handling a *continuous live feed* (data arriving piece by piece), which mimics how a real engine would transmit readings while flying. It's a clean demo moment and proves streaming capability — clearly labeled simulated so no one mistakes it for real telemetry.

The one piece that ties it all together (behind the scenes)

`predictRUL()` — a single function every feature calls to get a prediction. Right now it's a stand-in that fakes the numbers using our real test data, so the whole dashboard works with no backend. Later, we replace that one function with a call to your live model, and *nothing else changes*. That's the deliberate design: build and polish everything now, plug the real model in at the end.